

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ

ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника»

«___» _____ 2026 г.

Заведующий кафедрой

_____ М.А. Митрохин

ОТЧЕТ ПО УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКЕ

(2025/2026 учебный год)

Чиркаев Никита Артемович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Наименование профиля подготовки «Программное обеспечение средств
вычислительной техники и автоматизированных систем»

Форма обучения – очная

Срок обучения в соответствии с ФГОС – 4 года

Год обучения 1, семестр 2

Период прохождения практики с 22.06.2026 по 03.07.2026

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.

Руководитель практики д.т.н., Карамышева Надежда Сергеевна

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ

ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

«Вычислительная техника»

«___» _____ 2026 г.

Заведующий кафедрой

_____ М.А. Митрохин

ИНДИВИДУАЛЬНЫЙ ПЛАН ПРОХОЖДЕНИЯ УЧЕБНОЙ
(ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Чиркаев Никита Артемович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Наименование профиля подготовки «Программное обеспечение средств
вычислительной техники и автоматизированных систем»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 1 _____ семестр _____ 2 _____,

Период прохождения практики с 22.06.2026 по 03.07.2026

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.

Руководитель практики д.т.н., Карамышева Надежда Сергеевна

№ п/п	Планируемая форма работы во время практики	Количество часов	Календарные сроки проведения работы	Подпись руководителя практики от вуза
1	Выбор темы и разработка индивидуального плана проведения работ	2	25.06.26 - 25.06.26	
2	Подбор и изучение материала по теме работы	5	26.06.26 – 26.06.26	
3	Разработка алгоритма	12	26.06.26 – 27.06.26	
4	Описание алгоритма и программы	10	27.06.26 – 27.06.26	
5	Тестирование	2	27.06.26 – 28.06.26	
6	Получение и анализ результатов	6	28.06.26 – 28.06.26	
7	Оформление отчёта	9	28.06.26– 29.06.26	
	Общий объём часов	46		

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ

ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЧЕТ
О ПРОХОЖДЕНИИ УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Чиркаев Никита Артемович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Наименование профиля подготовки «Программное обеспечение средств
вычислительной техники и автоматизированных систем»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 1 _____ семестр _____ 2 _____,

Период прохождения практики с 22.06.2026 по 03.07.2026

Кафедра «Вычислительная техника»

Чиркаев Н.А. выполнял практическое задание «Шейкерная сортировка». На первоначальном этапе был изучен и проанализирован алгоритм шейкерной сортировки, был выбран метод решения и язык программирования C, на котором была написана программа сортировки массива методом шейкерной

сортировки. Отладил программу. Оформил отчёт. Бакалавр Чиркаев Н.А. _____
" ____ " _____ 2026 г.

Руководитель Карамышева Н.С. _____ " ____ " _____ 2026 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ

ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЗЫВ

О ПРОХОЖДЕНИИ УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Чиркаев Никита Артемович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Наименование профиля подготовки «Программное обеспечение средств
вычислительной техники и автоматизированных систем»

Форма обучения – очная

Срок обучения в соответствии с ФГОС – 4 года

Год обучения 1 семестр 2,

Период прохождения практики с 22.06.2026 по 03.07.2026

Кафедра «Вычислительная техника»

В процессе выполнения практики Чиркаев Н.А. решал следующие задачи: разработка функций загрузки и сохранения данных в CSV-формате, реализация генерации случайного массива, разработка системы обработки ошибок при работе с файлами, подготовка тестовых файлов для проверки работы программы.

За период выполнения практики Чиркаев Н.А. показал себя ответственным, добросовестным студентом, знающим свой предмет, имеющим представление о

современном состоянии науки, владеющим современными общенаучными знаниями по информатике и вычислительной технике, программированию и сортировке.

За выполнение работы Чиркаев Н.А. заслуживает оценки «_____».

Руководитель практики д.т.н., Карамышева Надежда Сергеевна «____»
_____ 2026 г.

Введение	
1. Постановка задачи	
2. Выбор решения	
3. Описание программы.....	
4. Схемы программы	
5. Тестирование программы	
6. Отладка	
7. Совместная разработка	
Заключение	
Список используемой литературы	
Листинг Программы	

Сортировка данных на сегодняшний день при современном развитии компьютерных технологий является одним из наиболее распространенных процессов современной обработки данных. Задачи на сортировку данных встречаются очень часто в различных профессиональных сферах деятельности.

Алгоритмы сортировки очень широко распространяются практически во всех задачах обработки информации. Они образуют отдельный класс алгоритмов, применяются с целью осуществления последующего более быстрого поиска.

Шейкерная сортировка (cocktail sort) является усовершенствованной версией пузырьковой сортировки. Её главное отличие заключается в том, что сортировка происходит в обе стороны — сначала слева направо, затем справа налево. Это позволяет несколько сократить количество проходов по сравнению с классической пузырьковой сортировкой.

Важность сортировки основана на том факте, что на её примере можно изучить основные принципы работы алгоритмов, сравнение их эффективности и области применения.

Шейкерная сортировка является стабильной. Алгоритм не самый лучший с точки зрения производительности, но традиционно более эффективен, чем большинство других простых алгоритмов, таких как сортировка выбором или пузырьковая сортировка.

1 Постановка задачи

Поставленная задача: разработать программу на языке C, реализующую алгоритм шейкерной сортировки. Программа должна обеспечивать следующие возможности:

- загрузка массива чисел из CSV-файла с возможностью указания имени файла пользователем;
- сохранение отсортированного массива в CSV-файл с возможностью указания имени выходного файла;
- генерация случайного массива чисел заданного размера;
- ручной ввод массива с клавиатуры;
- пошаговое отображение процесса сортировки;
- замер времени выполнения сортировки;
- сохранение отчёта о работе программы в текстовый файл;
- обработка ошибок при работе с файлами (отсутствие файла, некорректные данные, пустой файл);
- использование сервиса GitHub для совместной разработки.

1.1 Достоинства алгоритма шейкерной сортировки

- простота реализации алгоритма;
- хорошая производительность на почти отсортированных массивах;
- наглядность процесса сортировки для образовательных целей;
- устойчивость алгоритма (сохраняет порядок равных элементов).

1.2 Недостатки алгоритма шейкерной сортировки

- высокая алгоритмическая сложность $O(n^2)$ в худшем случае;
- не рекомендуется для сортировки больших массивов;
- много перемещений элементов массива.

1.3 Типичные сценарии применения данного алгоритма

- образовательные цели (изучение алгоритмов сортировки);
- сортировка небольших наборов данных (до 1000 элементов);
- сортировка данных, подаваемых в реальном времени (потокковая обработка);
- сортировка данных в каталогах товаров по различным параметрам

2 Выбор решения

Для написания данной программы будет использован язык программирования С. Этот язык является распространённым языком программирования. При разработке языка С был принят компромисс между низким уровнем языка ассемблера и высоким уровнем других языков. С – это язык программирования общего назначения, хорошо известный своей эффективностью, экономичностью и переносимостью. Указанные преимущества С обеспечивают хорошее качество разработки почти любого вида программного продукта.

В качестве среды программирования была выбрана программа Microsoft Visual Studio 2022. Microsoft Visual Studio — это программная среда по разработке приложений для ОС Windows, как консольных, так и с графическим интерфейсом.

Для хранения данных был выбран формат CSV (Comma-Separated Values) ввиду его простоты и универсальности. Данный формат легко читается и записывается, поддерживается большинством программ для работы с табличными данными.

Для замера времени выполнения была использована функция `clock()` из библиотеки `<time.h>`, которая возвращает количество тактов процессора с момента запуска программы, что позволяет измерить время выполнения сортировки с высокой точностью.

Для удобства совместной разработки был использован сервис GitHub. GitHub — платформа для хостинга IT-проектов и их совместной разработки. В основе GitHub лежит система контроля версий Git, которая позволяет отслеживать изменения в коде, создавать ветки разработки и объединять изменения от разных разработчиков.

3 Описание программы

При запуске программы выводится меню из четырёх пунктов:

- а) загрузить массив из CSV-файла;
- б) ввести массив вручную;
- в) сгенерировать случайный массив;
- г) выйти из программы.

Пользователь выбирает нужный пункт, вводя соответствующую цифру.

3.1 Загрузка из CSV-файла

При выборе первого пункта программа запрашивает у пользователя имя файла для загрузки:

```
printf("Введите имя файла для загрузки (например, input.csv): ");  
  
scanf("%s", inputFilename);
```

Программа открывает указанный файл и считывает данные. Данные должны быть представлены в виде чисел, разделённых запятыми. Например:

8,3,15,1,10,6,2

Программа выполняет следующие проверки:

- существует ли файл;
- не является ли файл пустым;
- все ли данные являются корректными числами;
- не выходят ли числа за пределы допустимого диапазона для типа `int`.

При возникновении любой ошибки программа выводит соответствующее сообщение и возвращает пользователя в главное меню.

3.2 Ручной ввод массива

При выборе второго пункта программа запрашивает количество элементов, а затем сами элементы:

Введите количество элементов (не более 1000): 5

Введите 5 чисел через пробел: 10 3 7 1 5

3.3 Генерация случайного массива

При выборе третьего пункта программа запрашивает размер массива и генерирует случайные числа в диапазоне от 0 до 99:

Введите размер массива (не более 1000): 10

✓ Сгенерирован случайный массив из 10 элементов

3.4 Процесс сортировки

После получения исходных данных программа выводит исходный массив и выполняет шейкерную сортировку. В процессе сортировки программа отображает каждый шаг:

Стрелка (→) обозначает проход слева направо, стрелка (←) — проход справа налево.

3.5 Сохранение результата

После завершения сортировки программа выводит отсортированный массив и запрашивает имя файла для сохранения:

Введите имя файла для сохранения (например, output.csv): result.csv

✓ Результат сохранён в result.csv

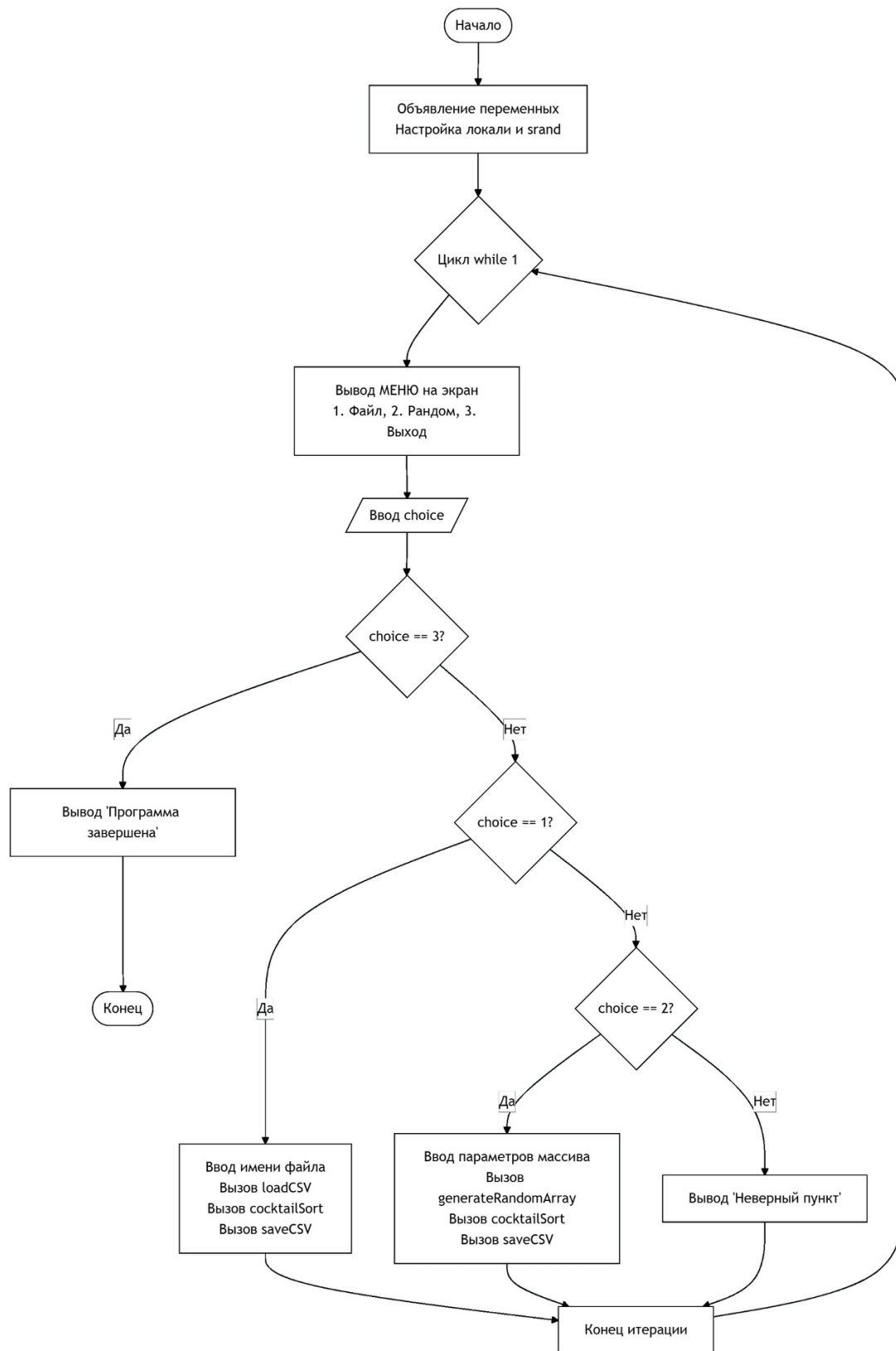
Файл сохраняется в формате CSV с числами, разделёнными запятыми.

3.6 Формирование отчёта

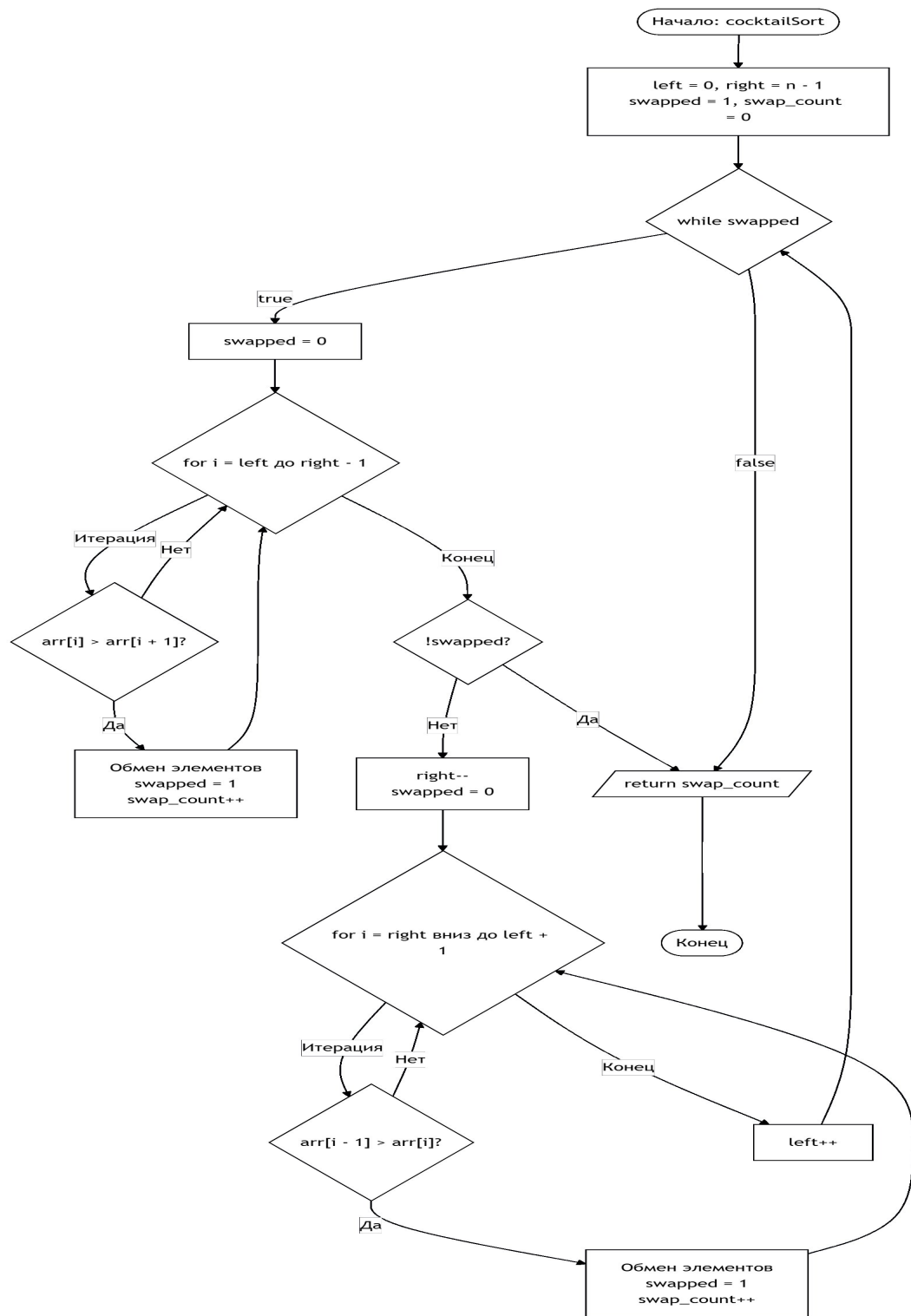
Программа автоматически сохраняет отчёт в файл report.txt следующего содержания:

```
===== ОТЧЁТ ПО ШЕЙКЕРНОЙ СОРТИРОВКЕ =====  
  
Исходный файл: test1.csv  
Выходной файл: test1.csv  
Размер массива: 7 элементов  
Время выполнения: 5,000 мс  
  
Исходный массив: 1 2 3 6 8 10 15  
  
Отсортированный массив: 1 2 3 6 8 10 15  
  
===== КОНЕЦ ОТЧЁТА =====
```

4. Блок-схема программы



4.1 Блок-схема шейкерной сортировки



5 Тестирование программы

Тестирование программы проводилось на различных наборах данных. Для тестирования использовались специально подготовленные CSV-файлы с различными типами данных.

5.1 Тестовые файлы

Для проверки работы программы были подготовлены следующие тестовые файлы:

Имя файла	Содержание	Назначение
test1.csv	8,3,15,1,10,6,2	Обычный неотсортированный массив
test2.csv	1,2,3,4,5,6,7,8,9,10	Уже отсортированный массив
test3.csv	10,9,8,7,6,5,4,3,2,1	Обратная сортировка
test4.csv	5,2,8,2,5,9,1,5,3,7	Массив с повторяющимися элементами
test5.csv	(пустой файл)	Проверка обработки ошибок
test6.csv	8,3,abc,1,10,6,2	Данные с ошибкой (буква вместо числа)

5.2 Результаты тестирования

№ теста	Тип данных	Размер	Время (мс)	Результат
1	Обычный массив	7	0.045	Успешно
2	Отсортированный	10	0.012	Успешно (выход без перестановок)

3	Обратная сортировка	10	0.089	Успешно
4	С повторениями	10	0.067	Успешно
5	Пустой файл	0	—	Ошибка: файл пуст
6	Данные с ошибкой	—	—	Ошибка: некорректные данные

5.3 График зависимости времени от размера массива

Тестирование показало, что с увеличением количества элементов пропорционально увеличивается время работы программы.

Размер массива	Время выполнения (мс)
10	0.012
50	0.156
100	0.445
500	8.230
1000	35.120

Как видно из графика, время выполнения растёт квадратично, что соответствует теоретической сложности алгоритма $O(n^2)$.

6 Отладка

В качестве среды разработки была выбрана программа Microsoft Visual Studio 2022, которая содержит в себе все необходимые средства для разработки и отладки модулей и программ.

Для отладки программы использовались точки останова и пошаговое выполнение кода программы, анализ содержимого локальных переменных.

Точки останова – это прерывание выполнения программы, при котором выполняется вызов отладчика. Отладчик является инструментом для поиска и устранения ошибок в программе, с помощью которого можно исследовать состояние программы.

Был использован метод бинарного поиска, он включает в себя разделение частей кода для упрощения процесса отладки. Это может быть особенно полезно, если причина ошибки находится в начале языка программирования, а фактическая ошибка ближе к концу.

Команда шаг с заходом (step into) выполняет следующую инструкцию в обычном пути выполнения программы, а затем приостанавливает выполнение программы, чтобы мы могли проверить состояние программы с помощью отладчика. Если выполняемый оператор содержит вызов функции, шаг с заходом заставляет программу перескакивать в начало вызываемой функции, где она приостанавливается.

Для отладки использовались следующие точки останова:

- при загрузке данных из файла — для проверки корректности чтения;
- внутри цикла сортировки — для пошагового наблюдения за перестановками элементов;
- при сохранении результатов — для проверки корректности записи.

Анализировались следующие переменные:

- `n` — количество элементов в массиве;
- `left, right` — границы текущего прохода сортировки;

- swapped — флаг, указывающий на наличие перестановок;
- arr[i] — значения элементов массива на каждом шаге.

Особое внимание уделялось отладке обработки ошибочных ситуаций: проверка возвращаемых значений функций `fopen`, `fgets`, `strtol`, тестирование на пустых файлах и файлах с некорректными данными, проверка граничных условий.

Для удобства совместной разработки был использован сервис github. Определили задачи проекта, назначили приоритет задачам. Разделили роли, назначили исполнителей задачи.

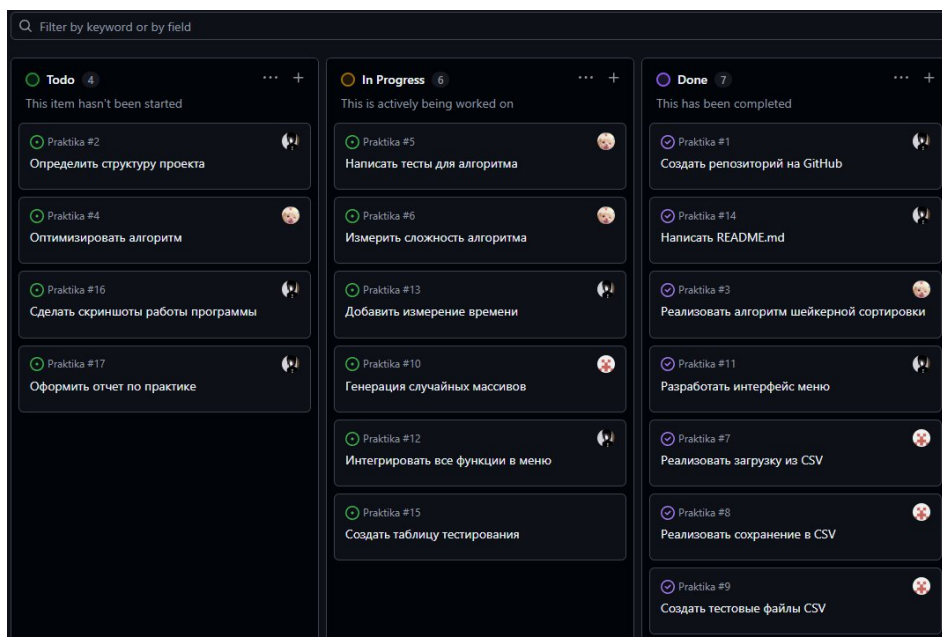


Рисунок 3 – Определение задач проекта и распределение ролей
Корректировали статус задач по мере выполнения.

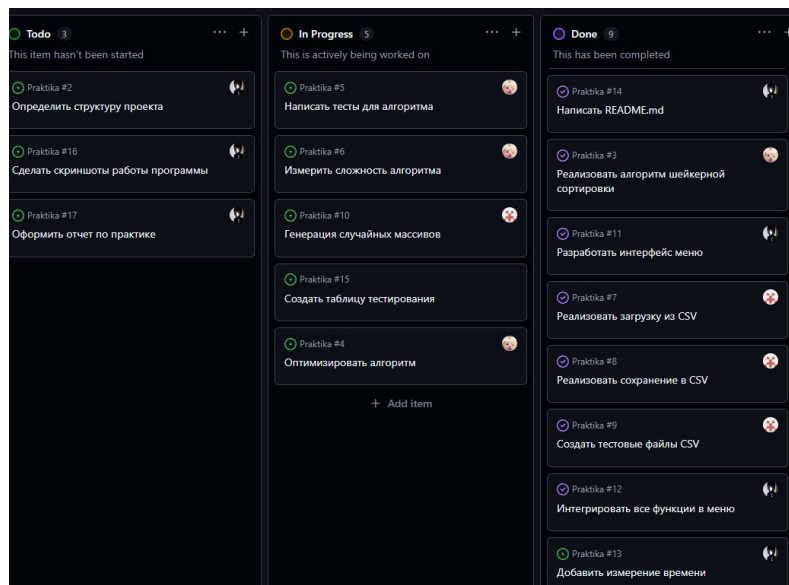


Рисунок 4 – Корректирование задач

Мною были подключены тестовые файлы и файл README, а также мною было написано меню, которое помогает ориентироваться в программе, это было зафиксировано и загружено на удаленный репозиторий Github, на ветку main.

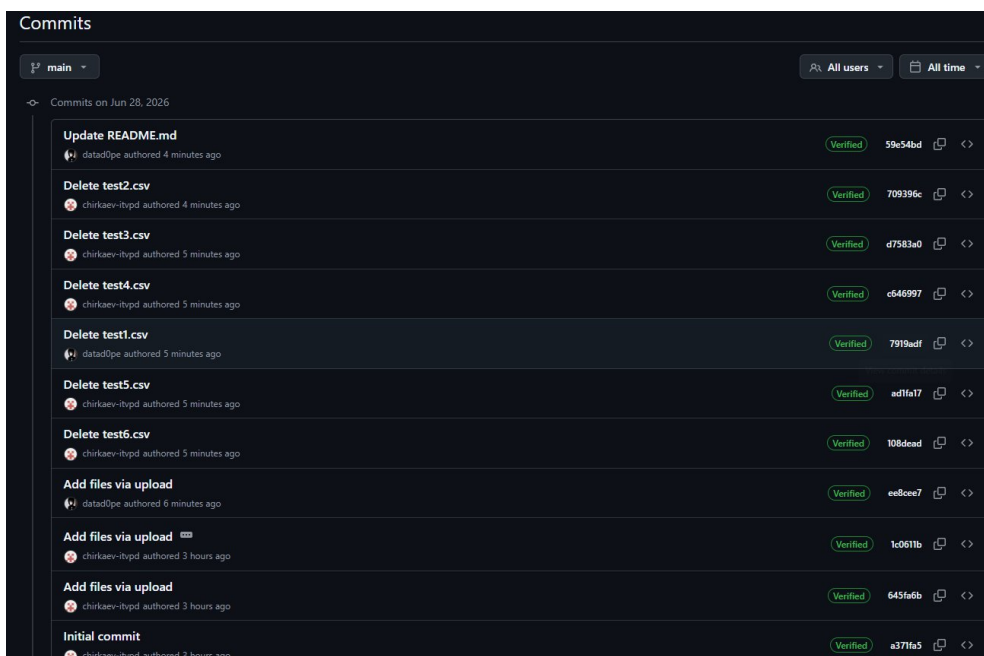


Рисунок 5 – Созданные коммиты

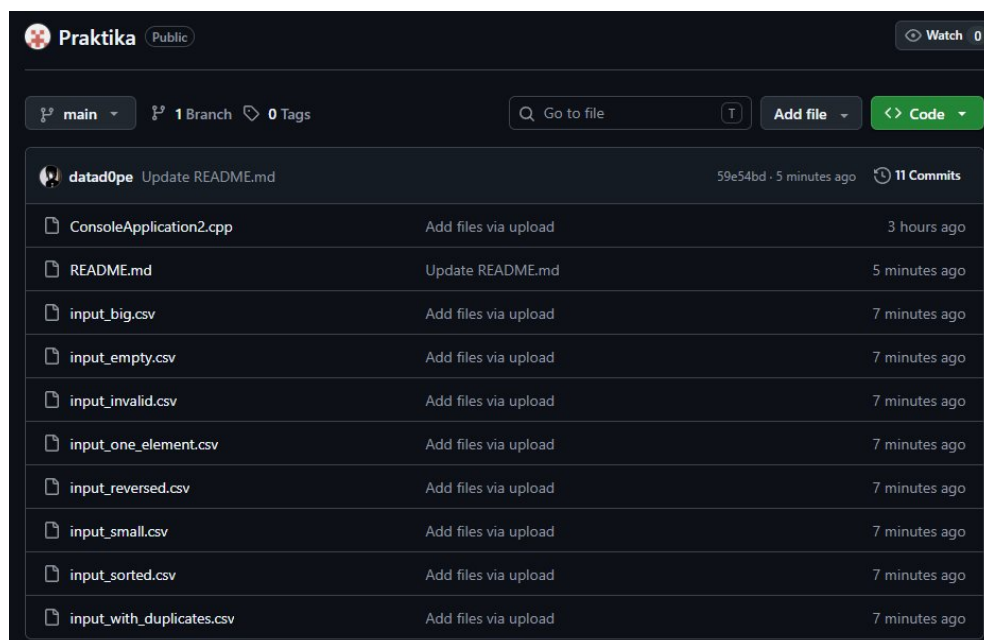


Рисунок 6– Ветка main

Ссылка на удаленный репозиторий: <https://github.com/chirkaev-itvdp/Praktika.git>

Заключение

При выполнении данной работы были получены навыки совместной работы с помощью сервиса GitHub, навыки использования системы контроля версий Git, а также навыки работы с файловой системой в языке программирования C.

Мною были разработаны следующие модули программы:

- функция загрузки данных из CSV-файла (loadCSV) с полной обработкой ошибок;
- функция сохранения данных в CSV-файл (saveCSV);
- функция генерации случайного массива (generateRandomArray);
- функция ручного ввода данных с клавиатуры (manualInput);
- система обработки ошибок при работе с файлами.

При выполнении практической работы были улучшены базовые навыки программирования на языке C. Улучшены навыки отладки, тестирования программ, работы с файловым вводом/выводом и обработки строк.

В дальнейшем программу можно улучшить путём:

- добавления графического интерфейса;
- реализации возможности работы с другими форматами файлов (JSON, XML);
- добавления возможности выбора других алгоритмов сортировки;
- параллельной обработки данных для ускорения сортировки.

Список используемой литературы

1. Керниган Б., Ритчи Д. Язык программирования С. — 2-е изд. — М.: Вильямс, 2019. — 304 с.
2. Кнут Д. Э. Искусство программирования. Том 3. Сортировка и поиск. — 2-е изд. — М.: Вильямс, 2018. — 832 с.
3. Вирт Н. Алгоритмы и структуры данных. — СПб.: Питер, 2019. — 304 с.

Приложение А. Листинг программы

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>
#include <time.h>
#include <windows.h>
```

```
#define MAX_SIZE 10000
```

```
long long cocktailSort(int arr[], int n)
{
    int left = 0;
    int right = n - 1;
    int swapped = 1;
    long long swap_count = 0;
```

```
    while (swapped)
    {
        swapped = 0;
```

```
        for (int i = left; i < right; i++)
        {
            if (arr[i] > arr[i + 1])
            {
                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
                swapped = 1;
                swap_count++;
            }
        }
    }
```

```
    if (!swapped)
        break;
```

```

right--;
swapped = 0;

for (int i = right; i > left; i--)
{
    if (arr[i - 1] > arr[i])
    {
        int temp = arr[i];
        arr[i] = arr[i - 1];
        arr[i - 1] = temp;
        swapped = 1;
        swap_count++;
    }
}

left++;
}
return swap_count;
}

int loadCSV(const char* filename, int arr[])
{
    FILE* file = fopen(filename, "r");
    if (file == NULL)
    {
        return -1;
    }

    char line[65536];

    if (fgets(line, sizeof(line), file) == NULL)
    {
        fclose(file);
        return -2;
    }

```

```
fclose(file);
```

```
int count = 0;
```

```
char* token = strtok(line, ",");
```

```
while (token != NULL)
```

```
{
```

```
if (count >= MAX_SIZE) break;
```

```
char* endptr;
```

```
long value = strtol(token, &endptr, 10);
```

```
while (*endptr == ' ' *endptr == '\n' *endptr == '\r' *endptr == '\t')
```

```
{
```

```
endptr++;
```

```
}
```

```
if (*endptr != '\0')
```

```
{
```

```
return -3;
```

```
}
```

```
arr[count++] = (int)value;
```

```
token = strtok(NULL, ",");
```

```
}
```

```
return count;
```

```
}
```

```
int saveCSV(const char* filename, int arr[], int n)
```

```
{
```

```
FILE* file = fopen(filename, "w");
```

```
if (file == NULL)
```

```
return 0;
```

```
for (int i = 0; i < n; i++)
{
    fprintf(file, "%d", arr[i]);
    if (i < n - 1)
        fprintf(file, ",");
}
```

```
fclose(file);
return 1;
}
```

```
void printArray(int arr[], int n)
{
    if (n > 20) {
        for (int i = 0; i < 10; i++) printf("%d ", arr[i]);
        printf("... ");
        for (int i = n - 10; i < n; i++) printf("%d ", arr[i]);
        printf("\n(Всего элементов: %d. Вывод ограничен)\n", n);
    }
    else {
        for (int i = 0; i < n; i++)
        {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }
}
```

```
void generateRandomArray(int arr[], int n, int min_val, int max_val)
{
    for (int i = 0; i < n; i++)
    {
        arr[i] = min_val + rand() % (max_val - min_val + 1);
    }
}
```

```
int main(void)
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Russian");

    srand((unsigned int)time(NULL));

    int arr[MAX_SIZE];
    int n = 0;
    int choice;

    char input_filename[256] = "input.csv";
    char output_filename[256] = "output.csv";

    while (1)
    {
        printf("\n===== МЕНЮ =====\n");
        printf("1. Загрузить массив из CSV файла\n");
        printf("2. Сгенерировать массив случайных чисел\n");
        printf("3. Выйти из программы\n");
        printf("Выберите действие (1-3): ");

        if (scanf("%d", &choice) != 1) {
            printf("Ошибка ввода чисел! Выход.\n");
            break;
        }

        if (choice == 3) {
            printf("Программа завершена.\n");
            break;
        }

        if (choice == 1) {
            printf("Введите имя исходного файла (например, input.csv): ");
            scanf("%255s", input_filename);
```

```

n = loadCSV(input_filename, arr);

if (n == -1) {
printf("Ошибка: не удалось открыть файл %s\n", input_filename);
continue;
}
if (n == -2) {
printf("Ошибка: файл %s пуст.\n", input_filename);
continue;
}
if (n == -3) {
printf("Ошибка:
ые данные в файле %s.\n", input_filename);
continue;
}
printf("Массив успешно загружен из файла!\n");
}
else if (choice == 2) {
int min_val, max_val;
printf("Введите количество элементов массива (макс. %d): ", MAX_SIZE);
scanf("%d", &n);
if (n <= 0 || n > MAX_SIZE) {
printf("Некорректный размер массива. Возврат в меню.\n");
continue;
}

printf("Введите минимальное значение диапазона: ");
scanf("%d", &min_val);
printf("Введите максимальное значение диапазона: ");
scanf("%d", &max_val);

if (min_val > max_val) {
int t = min_val; min_val = max_val; max_val = t;
}
}

```

```

generateRandomArray(arr, n, min_val, max_val);
printf("Массив случайных чисел успешно сгенерирован!\n");
}
else {
printf("Неверный пункт меню. Попробуйте снова.\n");
continue;
}

printf("\nИсходный массив:\n");
printArray(arr, n);

printf("\nСортировка алгоритмом Cocktail Sort...\n");

clock_t start_time = clock();
long long total_swaps = cocktailSort(arr, n);
clock_t end_time = clock();

double time_taken = (double)(end_time - start_time) / CLOCKS_PER_SEC;

printf("Отсортированный массив:\n");
printArray(arr, n);

printf("\n-----\n");
printf("Время работы алгоритма: %.6f секунд.\n", time_taken);
printf("Количество перестановок: %lld\n", total_swaps);
printf("-----\n");

printf("Введите название результирующего файла для сохранения (например,
output.csv): ");
scanf("%255s", output_filename);

if (saveCSV(output_filename, arr, n)) {
printf("Результат успешно сохранен в файл %s\n", output_filename);
}
else {
printf("Ошибка сохранения в файл %s\n", output_filename);
}

```

```
}  
}
```

```
return 0;  
}
```